EXPERIMENT – 04

Analyze and compare the text generation capabilities of different LLM architectures and access methods (OpenAI API, Hugging Face):

AIM:

To analyze and compare the text generation capabilities of different LLM architecture and access method.

ALGORITHM AND PROGRAM:

Step 1: Install & Import Dependencies.

```
!pip install transformers accelerate sentencepiece datasets evaluate rouge_score
bert_score -q
!pip install plotly kaleido -q
print("All packages installed!")
import torch
import time
import warnings
import numpy as np
import pandas as pd
from collections import Counter
from transformers import AutoTokenizer, AutoModelForCausalLM, pipeline
import plotly.graph_objects as go
import plotly.express as px
from plotly.subplots import make_subplots
import matplotlib.pyplot as plt
```

Step 2: Step 2: Define Models & Prompts.

```
MODEL_CONFIGS = [
    {
        "name": "GPT-2",
        "model_id": "gpt2",
        "architecture": "Decoder-only (Causal LM)",
        "params_M": 124,
        "family": "OpenAI GPT",
        "color": "#4F46E5"
    },
    {
```

"name": "DistilGPT2",
"model_id": "distilgpt2",
"architecture": "Distilled Decoder-only",
"params_M": 82,
"family": "HuggingFace Distil",
"color": "#7C3AED"
},
{
"name": "GPT-Neo 125M",
"model_id": "EleutherAI/gpt-neo-125m",
"architecture": "Decoder-only (Causal LM)",
"params_M": 125,
"family": "EleutherAI",
"color": "#059669"
},
{
"name": "OPT-125M",
"model_id": "facebook/opt-125m",
"architecture": "Decoder-only (Causal LM)",
"params_M": 125,
"family": "Meta OPT",
"color": "#DC2626"
},
{
"name": "BLOOM-560M",
"model_id": "bigscience/bloom-560m",
"architecture": "Decoder-only (ALiBi Attention)",
"params_M": 560,
"family": "BigScience",
"color": "#D97706"
},
]

PROMPTS = [
{
"id": "science",
"text": "The theory of relativity fundamentally changed our understanding of space and time because",
"category": "Scientific Explanation"
},
{
"id": "story",
"text": "Once upon a time in a distant galaxy, a young astronaut discovered a

```
mysterious signal that",
        "category": "Creative Writing"
    },
    {
        "id": "technical",
        "text": "Machine learning algorithms improve their performance over time by",
        "category": "Technical Description"
    },
    {
        "id": "opinion",
        "text": "The most significant challenge facing humanity in the 21st century is",
        "category": "Opinion / Open-ended"
    },
]

# Generation parameters
GEN_PARAMS = {
    "max_new_tokens": 100,
    "do_sample": True,
    "temperature": 0.8,
    "top_p": 0.92,
    "top_k": 50,
    "repetition_penalty": 1.2,
    "num_return_sequences": 1,
}

print(f"Models to evaluate : {len(MODEL_CONFIGS)}")
print(f"Prompts per model  : {len(PROMPTS)}")
print(f"Total generations  : {len(MODEL_CONFIGS) * len(PROMPTS)}")
print("\nModels:")
for m in MODEL_CONFIGS:
    print(f"  • {m['name']:15s} | {m['params_M']:>4}M params | {m['architecture']}")
```

Step 3: Metric Helper Functions.

```
def compute_distinct_n(text, n=2):
    """Distinct-N: ratio of unique n-grams to total n-grams (diversity measure)."""
    tokens = text.lower().split()
    if len(tokens) < n:
        return 0.0
    ngrams = [tuple(tokens[i:i+n]) for i in range(len(tokens)-n+1)]
    if not ngrams:
```

```
        return 0.0
    return len(set(ngrams)) / len(ngrams)

def compute_repetition_score(text):
    """Fraction of repeated 4-grams (lower = more repetitive)."""
    tokens = text.lower().split()
    if len(tokens) < 4:
        return 1.0
    fourgrams = [tuple(tokens[i:i+4]) for i in range(len(tokens)-3)]
    unique_ratio = len(set(fourgrams)) / len(fourgrams)
    return unique_ratio

def compute_avg_sentence_length(text):
    """Average words per sentence."""
    sentences = [s.strip() for s in text.replace('!','.').replace('?','.').split('.') if s.strip()]
    if not sentences:
        return 0
    lengths = [len(s.split()) for s in sentences]
    return np.mean(lengths)

def compute_vocabulary_richness(text):
    """Type-Token Ratio: unique words / total words."""
    tokens = text.lower().split()
    if not tokens:
        return 0.0
    return len(set(tokens)) / len(tokens)

def compute_perplexity(model, tokenizer, text, device, max_len=512):
    """Compute perplexity of the generated text under the model itself."""
    model.eval()
    encodings = tokenizer(text, return_tensors='pt', truncation=True,
max_length=max_len)
    input_ids = encodings.input_ids.to(device)
    with torch.no_grad():
        outputs = model(input_ids, labels=input_ids)
        loss = outputs.loss
    return torch.exp(loss).item()

print("Metric functions defined!")
```

Step 4: Load Models & Run Generation Benchmark.

```
results = []

for model_cfg in MODEL_CONFIGS:
    model_name = model_cfg['name']
    model_id   = model_cfg['model_id']

    print(f"\n{'='*60}")
    print(f"  Loading: {model_name}  ({model_id})")
    print(f"{'='*60}")

    try:

        tokenizer = AutoTokenizer.from_pretrained(model_id)
        if tokenizer.pad_token is None:
            tokenizer.pad_token = tokenizer.eos_token

        model = AutoModelForCausalLM.from_pretrained(
            model_id,
            torch_dtype=torch.float16 if device == 'cuda' else torch.float32,
            low_cpu_mem_usage=True,
        ).to(device)
        model.eval()

        param_count = sum(p.numel() for p in model.parameters()) / 1e6
        print(f"  ✓ Loaded | {param_count:.1f}M parameters")


        for prompt_cfg in PROMPTS:
            prompt_text = prompt_cfg['text']
            print(f"  Prompt [{prompt_cfg['category']}]... ", end="")

            inputs = tokenizer(prompt_text, return_tensors='pt',
                               padding=True, truncation=True).to(device)
            input_len = inputs['input_ids'].shape[1]

            # Time the generation
            t0 = time.time()
            with torch.no_grad():
                output_ids = model.generate(
                    **inputs,
                    pad_token_id=tokenizer.eos_token_id,
                    **GEN_PARAMS
```

```
    )
gen_time = time.time() - t0

# Decode only the NEW tokens
new_tokens = output_ids[0][input_len:]
generated_text = tokenizer.decode(new_tokens, skip_special_tokens=True)
full_text = prompt_text + " " + generated_text

n_new_tokens = len(new_tokens)
tokens_per_sec = n_new_tokens / gen_time if gen_time > 0 else 0

# Compute metrics
perplexity  = compute_perplexity(model, tokenizer, full_text, device)
distinct1   = compute_distinct_n(generated_text, 1)
distinct2   = compute_distinct_n(generated_text, 2)
repetition  = compute_repetition_score(generated_text)
avg_sentlen = compute_avg_sentence_length(generated_text)
vocab_rich  = compute_vocabulary_richness(generated_text)

results.append({
    # Identity
    "model_name"   : model_name,
    "model_id"     : model_id,
    "architecture" : model_cfg['architecture'],
    "family"       : model_cfg['family'],
    "params_M"     : param_count,
    "color"        : model_cfg['color'],
    # Prompt
    "prompt_id"    : prompt_cfg['id'],
    "category"     : prompt_cfg['category'],
    "prompt_text"  : prompt_text,
    # Generated output
    "generated_text": generated_text,
    # Speed
    "gen_time_sec" : round(gen_time, 3),
    "n_tokens"     : n_new_tokens,
    "tokens_per_sec": round(tokens_per_sec, 1),
    # Quality metrics
    "perplexity"   : round(perplexity, 2),
    "distinct_1"   : round(distinct1, 4),
    "distinct_2"   : round(distinct2, 4),
    "repetition"   : round(repetition, 4),
    "vocab_richness": round(vocab_rich, 4),
```

```
            "avg_sent_len" : round(avg_sentlen, 2),
        })

        print(f"✓ {n_new_tokens} tokens in {gen_time:.2f}s ({tokens_per_sec:.1f}
tok/s) | PPL={perplexity:.1f}")

    # Free GPU memory between models
    del model
    torch.cuda.empty_cache() if device == 'cuda' else None

  except Exception as e:
    print(f"  ✗ ERROR loading {model_name}: {e}")
    continue

df = pd.DataFrame(results)
print(f"\n Benchmark complete! {len(df)} total generations recorded.")
```

Step 5: Aggregate Metrics per Model.

```
# Aggregate across prompts
agg = df.groupby('model_name').agg(
  architecture   = ('architecture',  'first'),
  family         = ('family',        'first'),
  params_M       = ('params_M',      'first'),
  color          = ('color',         'first'),
  tokens_per_sec = ('tokens_per_sec','mean'),
  perplexity     = ('perplexity',    'mean'),
  distinct_1     = ('distinct_1',    'mean'),
  distinct_2     = ('distinct_2',    'mean'),
  repetition     = ('repetition',    'mean'),
  vocab_richness = ('vocab_richness','mean'),
  avg_sent_len   = ('avg_sent_len',  'mean'),
).reset_index().round(3)

# Compute a composite quality score (normalized, higher = better)
def normalize_col(col, higher_is_better=True):
  mn, mx = col.min(), col.max()
  if mx == mn: return pd.Series([0.5]*len(col), index=col.index)
```

```
    norm = (col - mn) / (mx - mn)
    return norm if higher_is_better else 1 - norm

agg['score_speed']    = normalize_col(agg['tokens_per_sec'],  True)
agg['score_fluency']  = normalize_col(agg['perplexity'],      False)  # lower ppl =
better
agg['score_diversity']= normalize_col(agg['distinct_2'],      True)
agg['score_novelty']  = normalize_col(agg['repetition'],      True)
agg['score_vocab']    = normalize_col(agg['vocab_richness'],  True)

agg['composite_score'] = (
    0.20 * agg['score_speed'] +
    0.30 * agg['score_fluency'] +
    0.20 * agg['score_diversity'] +
    0.15 * agg['score_novelty'] +
    0.15 * agg['score_vocab']
).round(3)

agg = agg.sort_values('composite_score',
ascending=False).reset_index(drop=True)

display_cols = ['model_name','params_M','tokens_per_sec','perplexity',
                'distinct_2','repetition','vocab_richness','composite_score']
print("\n Aggregated Results (sorted by composite score):")
display(agg[display_cols].rename(columns={
    'model_name':'Model','params_M':'Params(M)','tokens_per_sec':'Tok/s',
    'perplexity':'Perplexity','distinct_2':'Distinct-2',
    'repetition':'Repetition','vocab_richness':'Vocab Richness',
    'composite_score':'Score'
})
```

Step 6: Visualization — Metric Comparison Charts.

```
colors = agg['color'].tolist()
models = agg['model_name'].tolist()

fig = make_subplots(
    rows=2, cols=3,
    subplot_titles=[
        'Generation Speed (tokens/sec)',
        'Perplexity (lower = better)',
        'Distinct-2 Score (diversity)',
        'Repetition Score (higher = less repetitive)',
        'Vocabulary Richness',
        'Composite Quality Score'
    ],
    vertical_spacing=0.18,
    horizontal_spacing=0.1
)

metrics = [
    ('tokens_per_sec', 1, 1),
    ('perplexity',     1, 2),
    ('distinct_2',     1, 3),
    ('repetition',     2, 1),
    ('vocab_richness', 2, 2),
    ('composite_score',2, 3),
]

for col_name, row, col in metrics:
    fig.add_trace(
        go.Bar(
            x=models,
            y=agg[col_name],
            marker_color=colors,
            text=agg[col_name].round(2),
            textposition='outside',
            showlegend=False,
```

```
        ),
        row=row, col=col
    )

fig.update_layout(
    title_text="LLM Architecture Text Generation — Metric
Comparison",
    title_font_size=18,
    height=700,
    paper_bgcolor='#0F172A',
    plot_bgcolor='#1E293B',
    font=dict(color='white', size=11),
)
fig.update_xaxes(tickangle=-30, tickfont=dict(size=10))
fig.show()
```

Step 7: Radar Chart: Multi-Dimensional Comparison

```
radar_metrics =
['score_speed','score_fluency','score_diversity','score_nov
elty','score_vocab']
radar_labels  =
['Speed','Fluency','Diversity','Novelty','Vocab Richness']

fig_radar = go.Figure()

for _, row in agg.iterrows():
    values = [row[m] for m in radar_metrics]
    values += [values[0]]  # close the polygon
    labels = radar_labels + [radar_labels[0]]

    fig_radar.add_trace(go.Scatterpolar(
        r=values,
        theta=labels,
        fill='toself',
        name=row['model_name'],
        line=dict(color=row['color'], width=2),
        fillcolor=row['color'],
        opacity=0.25,
    ))
```

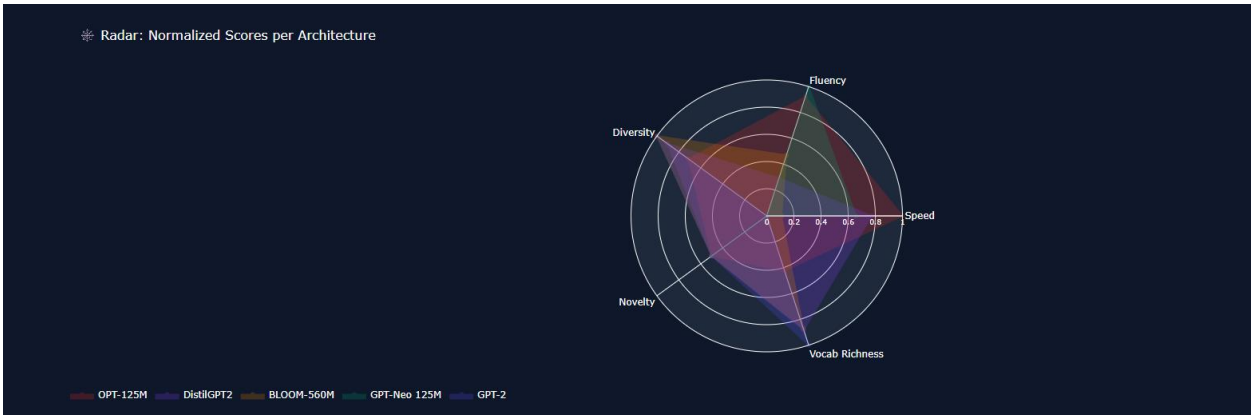
```
fig_radar.update_layout(
    polar=dict(
        radialaxis=dict(visible=True, range=[0,1],
tickfont=dict(size=9)),
        bgcolor='#1E293B'
    ),
    paper_bgcolor='#0F172A',
    font=dict(color='white'),
    title=' Radar: Normalized Scores per Architecture',
    title_font_size=16,
    legend=dict(orientation='h', yanchor='bottom', y=-0.2),
    height=520
)
fig_radar.show()
```

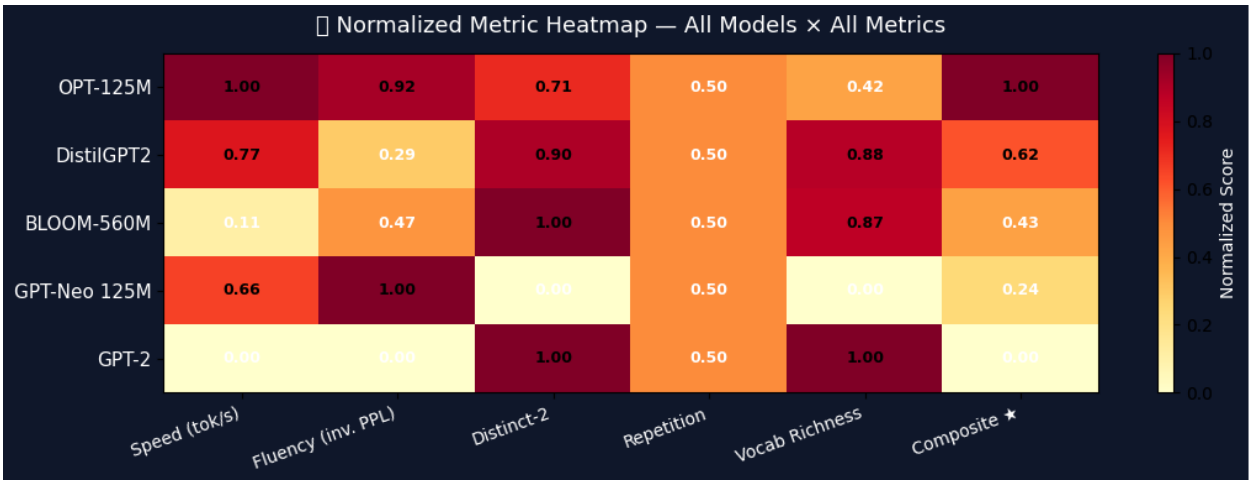
Step 8: Radar Chart:Multi-Dimensional Comparison.

```
import matplotlib.pyplot as plt
import matplotlib.colors as mcolors
import numpy as np

heatmap_metrics = {
    'Speed (tok/s)':      'tokens_per_sec',
    'Fluency (inv. PPL)': 'score_fluency',
    'Distinct-2':         'distinct_2',
    'Repetition':         'repetition',
    'Vocab Richness':     'vocab_richness',
    'Composite ':         'composite_score',
}

heat_data = pd.DataFrame(index=agg['model_name'])
for label, col in heatmap_metrics.items():
    vals = agg.set_index('model_name')[col]
    mn, mx = vals.min(), vals.max()
    heat_data[label] = ((vals - mn) / (mx - mn + 1e-
8)).round(3) if mx != mn else 0.5

fig_h, ax = plt.subplots(figsize=(11, 4))
fig_h.patch.set_facecolor('#0F172A')
ax.set_facecolor('#1E293B')

im = ax.imshow(heat_data.values, cmap='YlOrRd',
```

```
aspect='auto', vmin=0, vmax=1)

ax.set_xticks(range(len(heat_data.columns)))
ax.set_xticklabels(heat_data.columns, color='white',
fontsize=10, rotation=20, ha='right')
ax.set_yticks(range(len(heat_data.index)))
ax.set_yticklabels(heat_data.index, color='white',
fontsize=11)

for i in range(heat_data.shape[0]):
    for j in range(heat_data.shape[1]):
        val = heat_data.values[i, j]
        ax.text(j, i, f"{val:.2f}", ha='center', va='center',
                color='black' if val > 0.5 else 'white',
fontsize=9, fontweight='bold')

plt.colorbar(im, ax=ax, label='Normalized
Score').ax.yaxis.label.set_color('white')
ax.set_title(' Normalized Metric Heatmap — All Models
× All Metrics',
             color='white', fontsize=13, pad=12)
plt.tight_layout()
plt.savefig('llm_heatmap.png', dpi=150,
bbox_inches='tight', facecolor='#0F172A')
plt.show()
print(" Heatmap saved to llm_heatmap.png"
```

OUTPUT:





RESULT:

Thus, the program to analyze and compare the text generation capabilities of different LLM architecture and access method has been executed successfully.